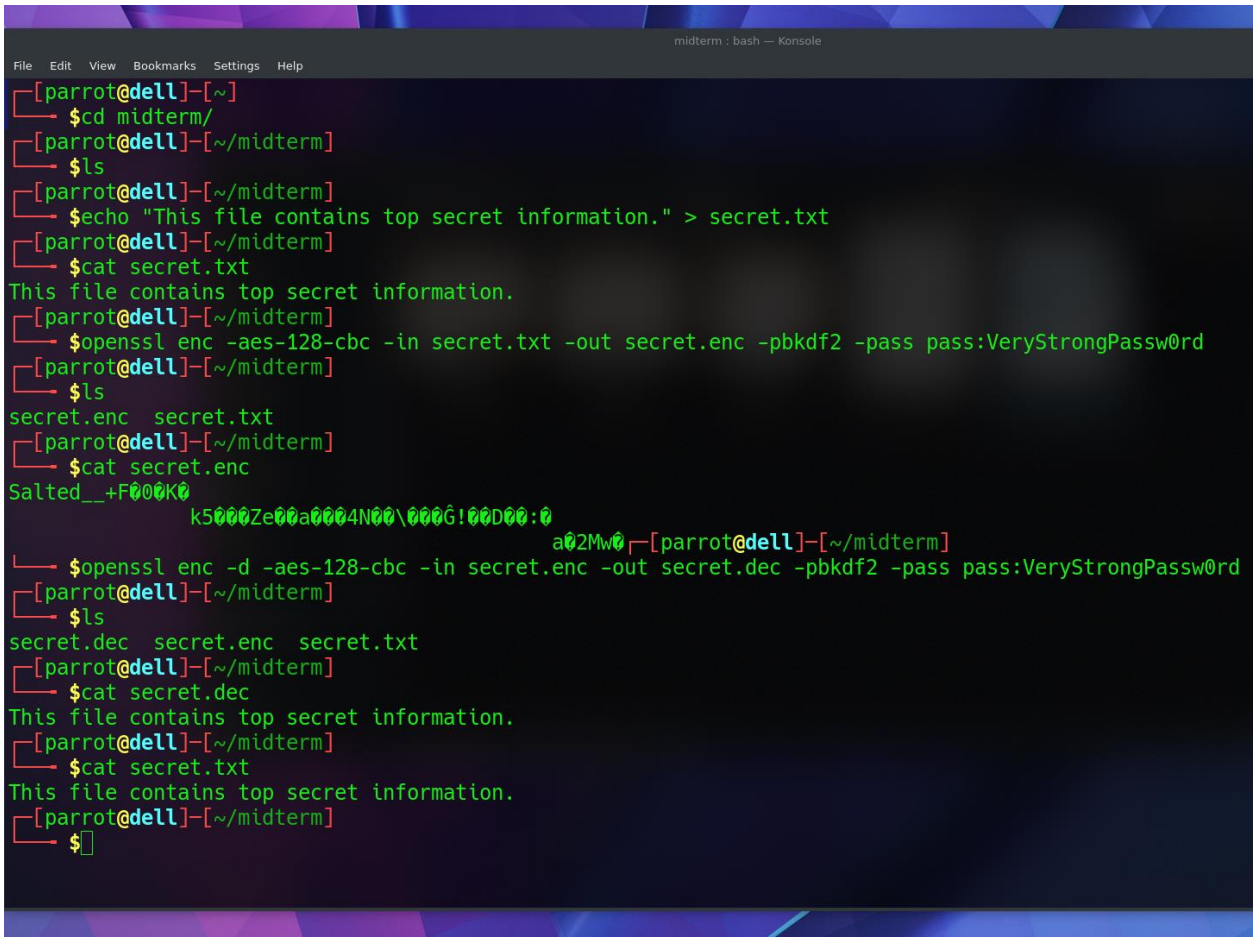
Applied Cryptography – Midterm Lab Exam (Weeks 1–4)

Task 1 – AES Encryption

Task 1B: Decrypt secret.enc



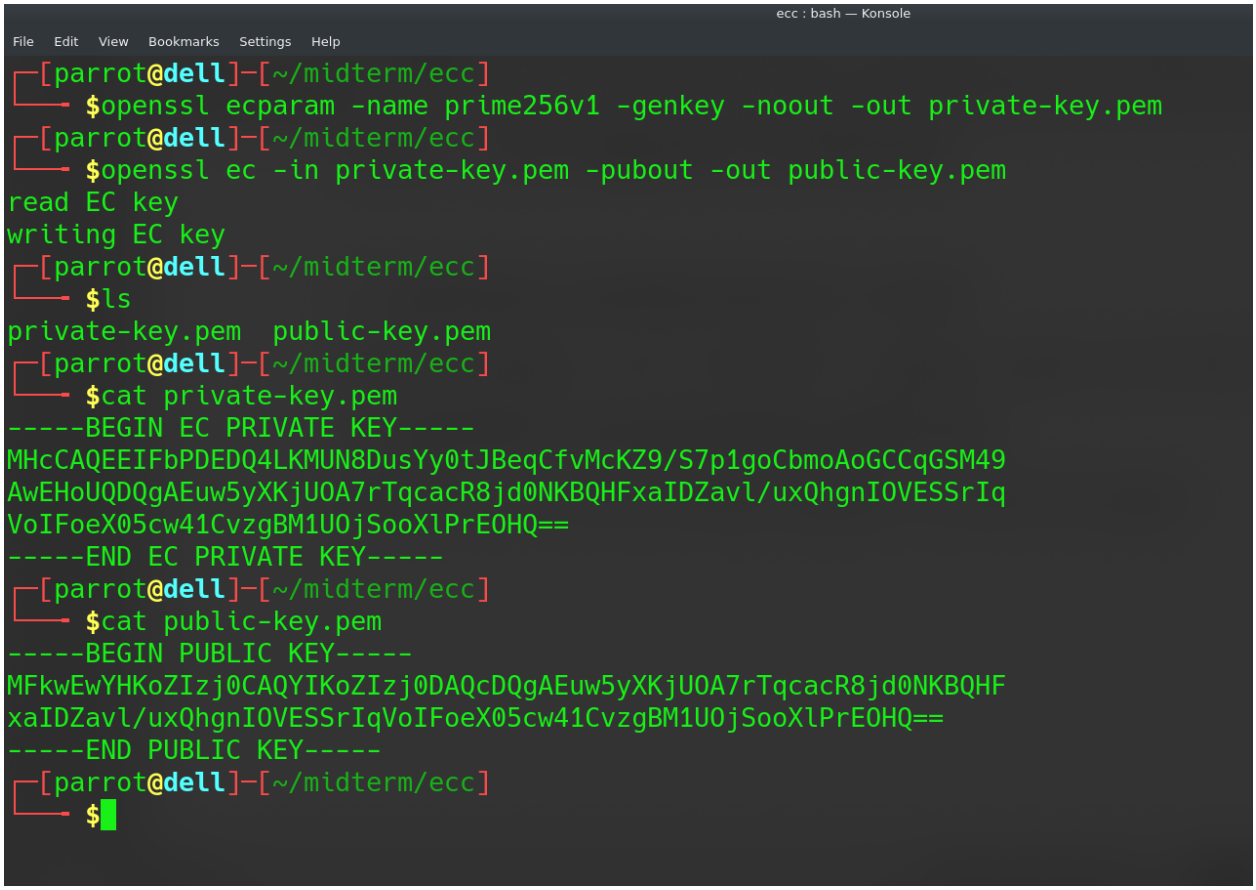
Task 2 - ECC Signature Verification

1 - generate a private key for a curve

openssl ecparam -name prime256v1 -genkey -noout -out private-key.pem

2 - generate corresponding public key

openssl ec -in private-key.pem -pubout -out public-key.pem



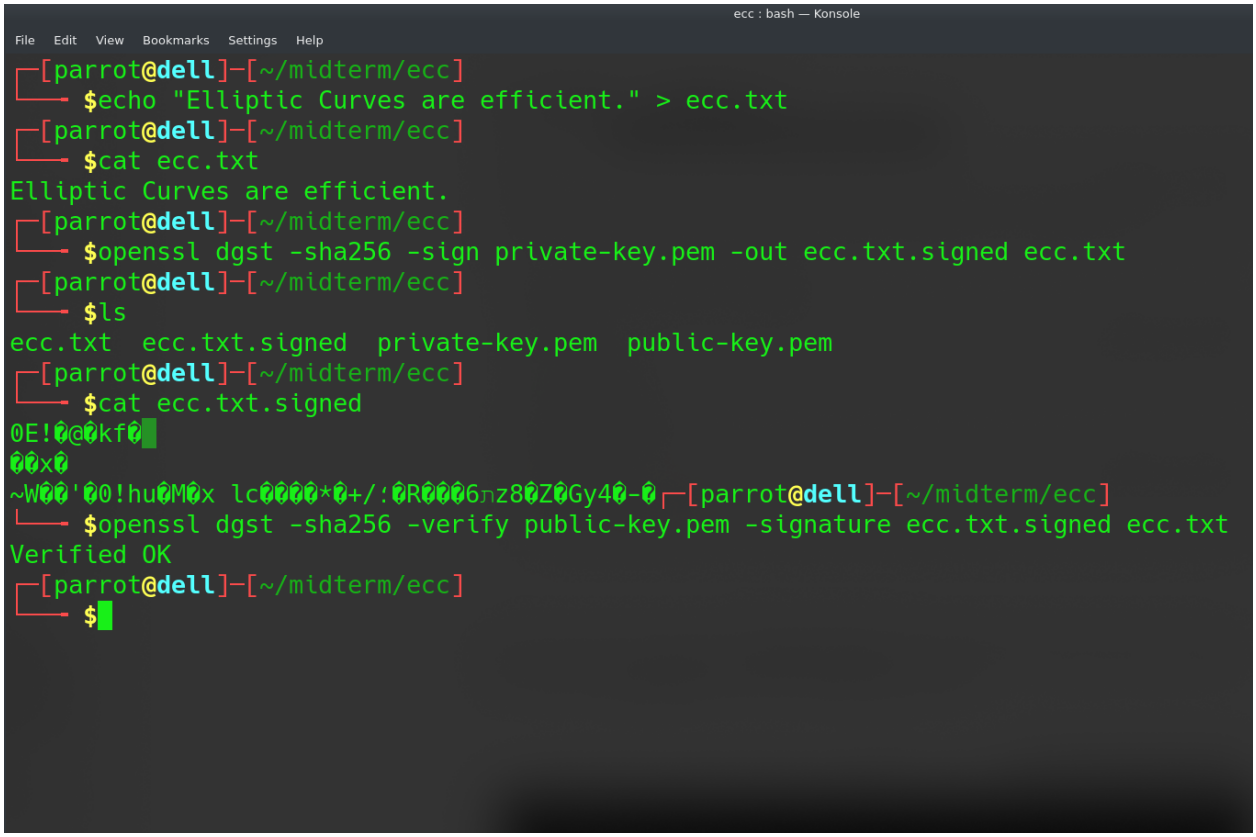
Task 2B: Sign and verify a message

3 - Let’s create digital signature for ecc.txt using private-key.pem

openssl dgst -sha256 -sign private-key.pem -out ecc.txt.signed ecc.txt

4 - And than verify the signature using the public-key.pem. If process will be successful, we’ll receive “Verified OK”

openssl dgst -sha256 -verify public-key.pem -signature ecc.txt.signed ecc.txt



Task 3 – Hashing & HMAC

1. Create data.txt with:

Never trust, always verify.

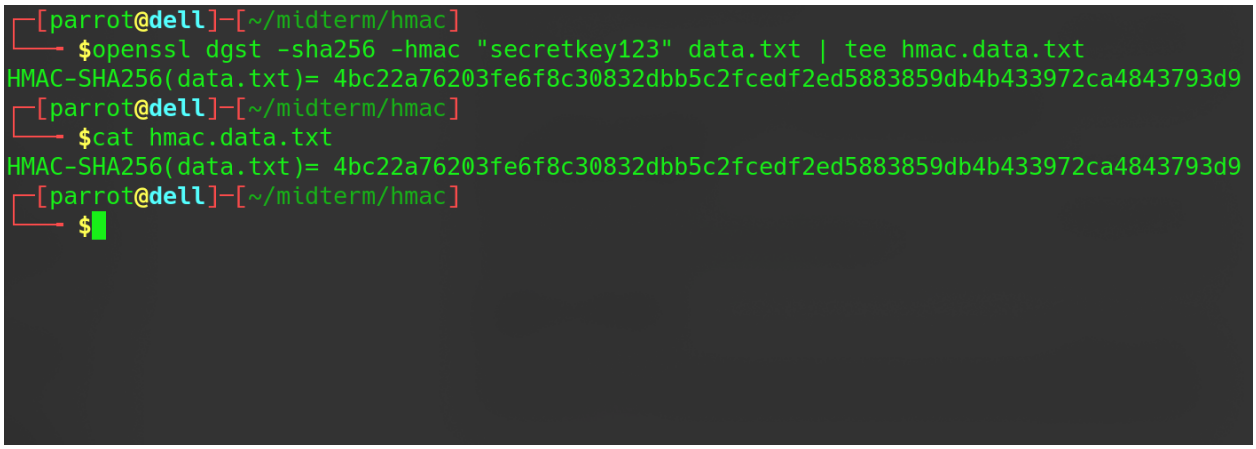
2. Hash it using Python or CLI.

3. Submit hash output and code/command.

Task 3B: HMAC using SHA-256

1. Use the key: secretkey123

2. Create an HMAC for data.txt.

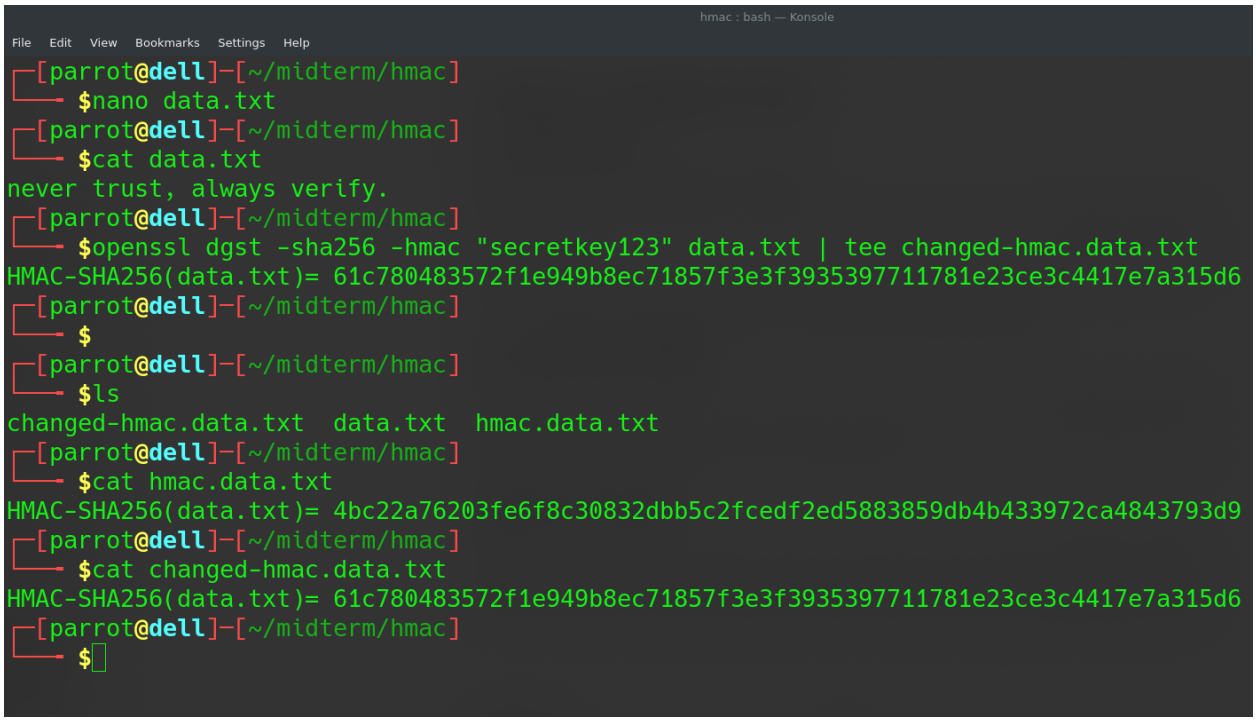


Task 3C: Integrity Check

1. Change one letter in data.txt.

(Changed upper-case “N” with lowercase “n”)

2. Recompute HMAC.



3. Explain what happens and why HMAC is important.

When we compute an HMAC (Hash-based Message Authentication Code), we are generating a cryptographic hash of your data combined with a secret key. This ensures both data integrity and authenticity.

When we later modified a single letter in the file from "Never" to "never" and recomputed the HMAC, the output was completely different. This dramatic change demonstrates a fundamental property of cryptographic hash functions: the avalanche effect, where even a tiny input change results in a vastly different output.

HMAC is crucial in secure communications because it allows the recipient to verify that the message was not tampered with and that it was generated by someone who knows the shared secret key. Unlike plain hashes (like SHA-256 alone), HMACs prevent attackers from simply replacing both the file and its hash. Without the key, they cannot forge a valid HMAC.

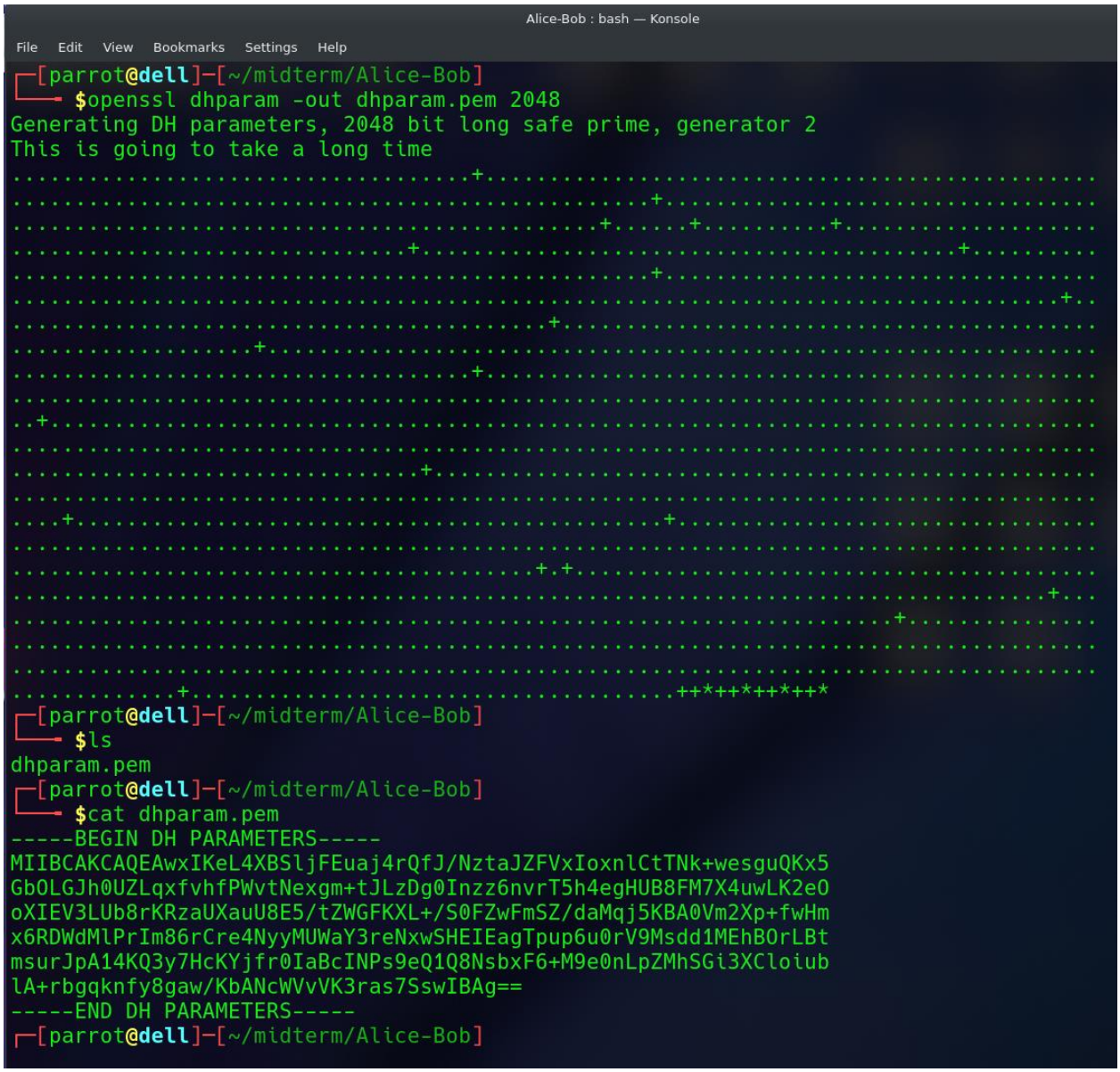
In real-world applications, HMACs are used in APIs, secure file transfers, and communication protocols like HTTPS or TLS. They provide a lightweight yet powerful mechanism to ensure that the data has not been altered in transit and is genuinely from the expected source.

Task 4 – Diffie-Hellman Key Exchange

1) Generate DH Parameters

These are the common parameters - both Alice and Bob will use.

openssl dhparam -out dhparam.pem 2048



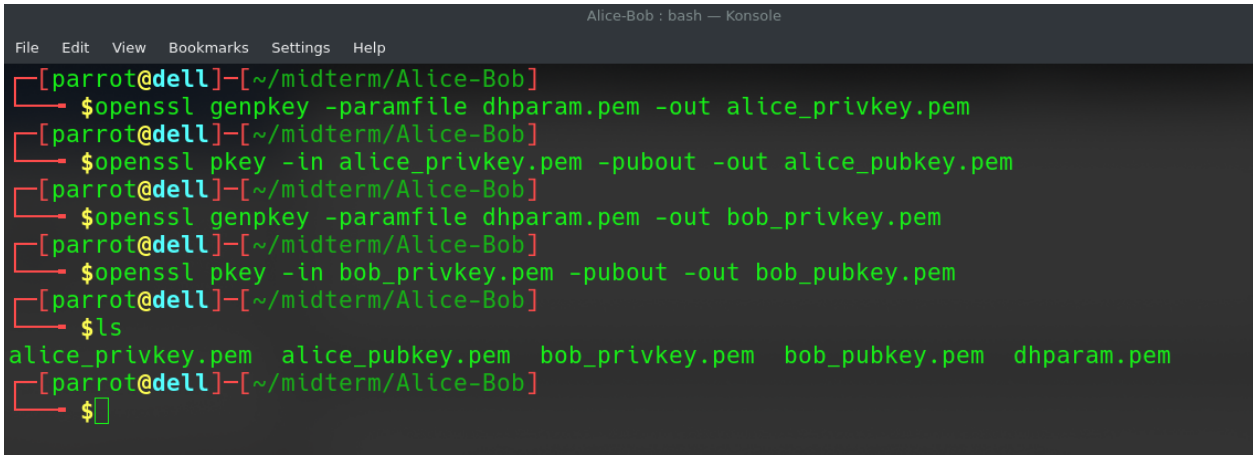
2) Generate Private and Public Keys for Alice and for Bob

openssl genpkey -paramfile dhparam.pem -out alice_privkey.pem

openssl pkey -in alice_privkey.pem -pubout -out alice_pubkey.pem

openssl genpkey -paramfile dhparam.pem -out bob_privkey.pem

openssl pkey -in bob_privkey.pem -pubout -out bob_pubkey.pem



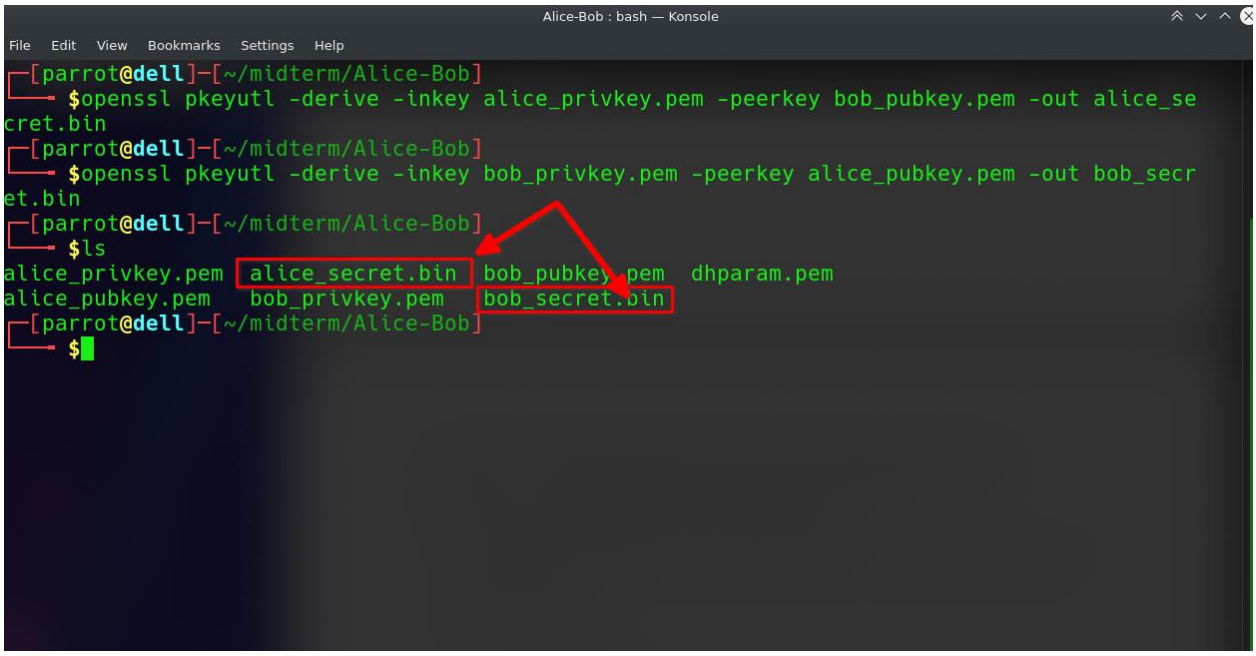
3) Derive Shared Secrets

Alice derives shared secret (using her privkey and Bob's pubkey):

openssl pkeyutl -derive -inkey alice_privkey.pem -peerkey bob_pubkey.pem -out alice_secret.bin

Bob derives shared secret (using his privkey and Alice's pubkey):

openssl pkeyutl -derive -inkey bob_privkey.pem -peerkey alice_pubkey.pem -out bob_secret.bin



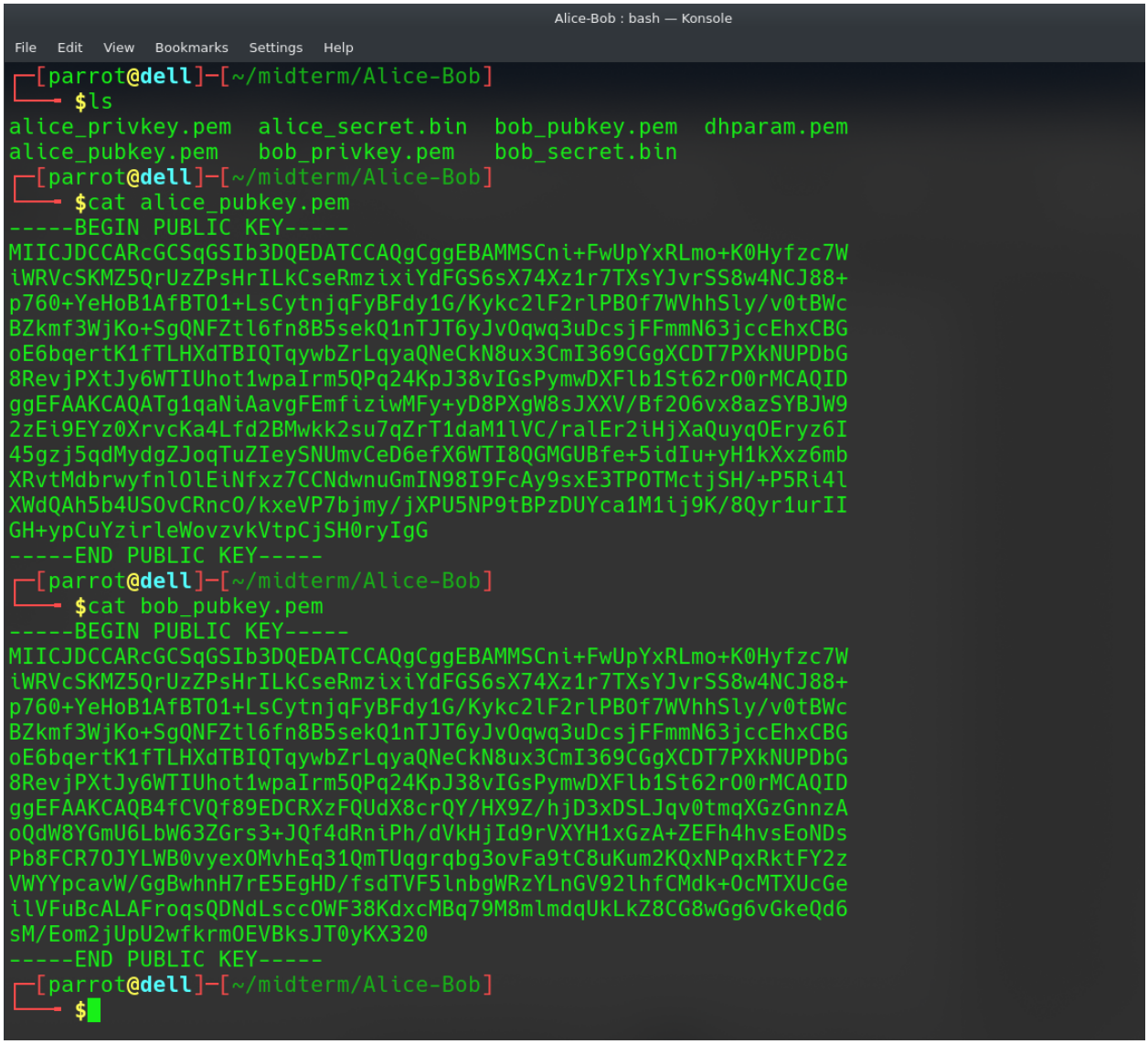
4) Check file contents

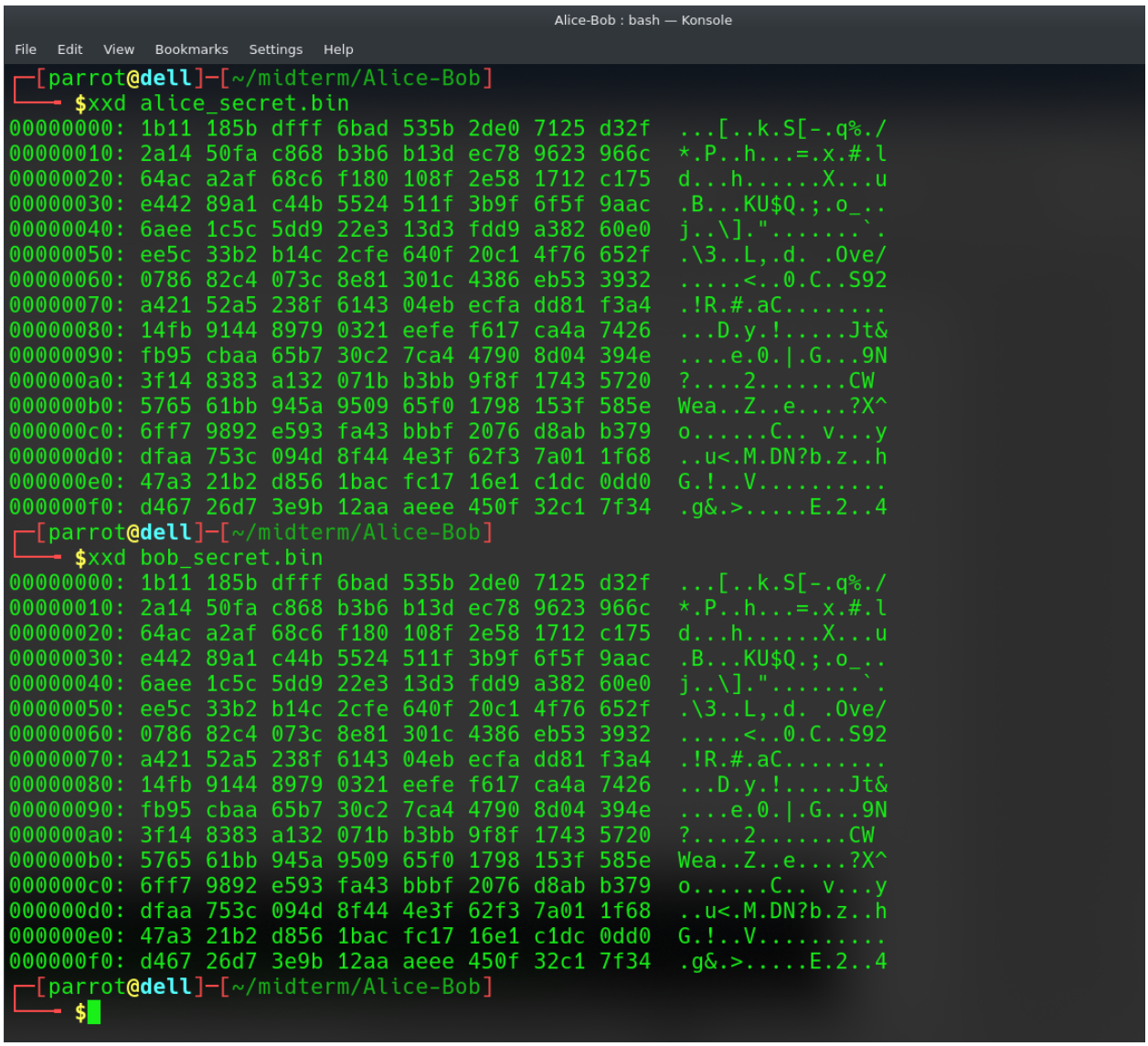
cat alice_pubkey.pem

cat bob_pubkey.pem

xxd alice_secret.bin

xxd bob_secret.bin





As we can see in the screenshot above, Alice and Bob's shared secret key contents are identical. They are now ready to continue exchanging information with each other using this shared key.

Task 4B: Real-Life Application

Diffie-Hellman (DH) is a foundational cryptographic algorithm used widely in secure communication systems to establish a shared secret between two parties over an insecure channel. One of its most common applications is in the TLS (Transport Layer Security) handshake, which secures web traffic. During a TLS handshake, ephemeral Diffie-Hellman (DHE) or elliptic-curve Diffie-Hellman (ECDHE) is used to create a temporary shared key between a client and server. This ensures forward secrecy, meaning even if the server's private key is compromised in the future, past communications remain secure. Another major use case is in secure messaging applications, such as Signal and WhatsApp, which implement the Signal Protocol. This protocol uses a variation called the Double Ratchet algorithm, which includes DH exchanges to generate and frequently refresh encryption keys. This ensures not only forward secrecy but also post-compromise security, meaning even if a device is compromised, future messages can still remain private.

The importance of Diffie-Hellman in secure communication lies in its ability to enable confidential key exchange without transmitting the actual key over the network. Unlike symmetric encryption, which requires both parties to already share a secret, DH allows secure key agreement even when the two sides have never communicated before. This makes it a cornerstone of modern cryptographic systems. Without such a method, all secure internet traffic, from online banking to private chats, would be vulnerable to eavesdropping and interception. By facilitating the safe exchange of encryption keys, Diffie-Hellman helps ensure data confidentiality, integrity, and trust in digital communication.

https://github.com/quazary/midterm.git